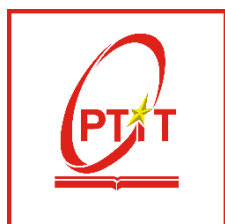


BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

-----o0o-----



ĐỒ ÁN CUỐI KỲ
PHÁT TRIỂN HỆ THỐNG CHAT
NGANG HÀNG P2P

Giảng viên hướng dẫn

Tên môn học

Nhóm môn học

Nhóm

Thành viên

: Kim Ngọc Bách

: Các hệ thống phân tán

: Nhóm 01

: Nhóm 11

: Nguyễn Khánh Đăng – B22DCCN209

Trần Duy Khởi – B22DCCN473

Bùi Quang Huy – B22DCCN377

Hà Nội – 2026

MỤC LỤC

MỤC LỤC	2
DANH MỤC HÌNH ẢNH	3
DANH MỤC BẢNG	4
CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN	5
CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG	6
2.1. Tổng quan kiến trúc	6
2.2. Các thành phần chính	6
2.3. Cơ sở dữ liệu	7
2.4. Công nghệ sử dụng	8
CHƯƠNG 3: GIAO THỨC TRAO ĐỔI THÔNG điệp	9
3.1. Cơ chế truyền thông	9
3.2. Bộ các loại message	10
3.3. Luồng trao đổi thông điệp chi tiết	12
3.4. Mã hóa nội dung	14
CHƯƠNG 4: CƠ CHẾ PEER DISCOVERY	15
4.1. Tổng quan	15
4.2. Quy trình Peer Discovery qua Bootstrap Server	15
4.3. Duy trì thông tin topology	15
4.4. Chống xử lý tin nhắn trùng	16
4.5. Kết nối Relay Server	16
CHƯƠNG 5: XỬ LÝ LỖI VÀ THỬ NGHIỆM HỆ THỐNG	17
5.1. Xử lý lỗi mạng — Tự động Reconnect	17
5.2. Xử lý lỗi database — Connection Pool	18
5.3. Xử lý lỗi truyền file	18
5.4. Unit Testing — BootstrapClientTest	18
5.5. Đánh giá hệ thống	19
CHƯƠNG 6: KẾT LUẬN	21

DANH MỤC HÌNH ẢNH

Hình 1: Kiến trúc tổng quan Chat P2P System	6
Hình 2: Luồng Peer đăng ký vào mạng.....	12
Hình 3: Luồng chat khi cả hai online	13
Hình 4: Luồng tin nhắn offline.....	13
Hình 5: Mã chống xử lý tin nhắn trùng lặp	16
Hình 6: Mã config cơ chế tự động reconnect.....	17

DANH MỤC BẢNG

Bảng 1: Bảng database hệ thống	7
Bảng 2: Công nghệ sử dụng	8
Bảng 3: Message nhóm Bootstrap.....	10
Bảng 4: Message nhóm Chat P2P	10
Bảng 5: Message nhóm Group Management.....	11
Bảng 6: Message nhóm File transfer.....	11
Bảng 7: Message nhóm Relay Server	12
Bảng 8: Các loại lỗi được xử lý	17
Bảng 9: HikariCP cấu hình timeout	18

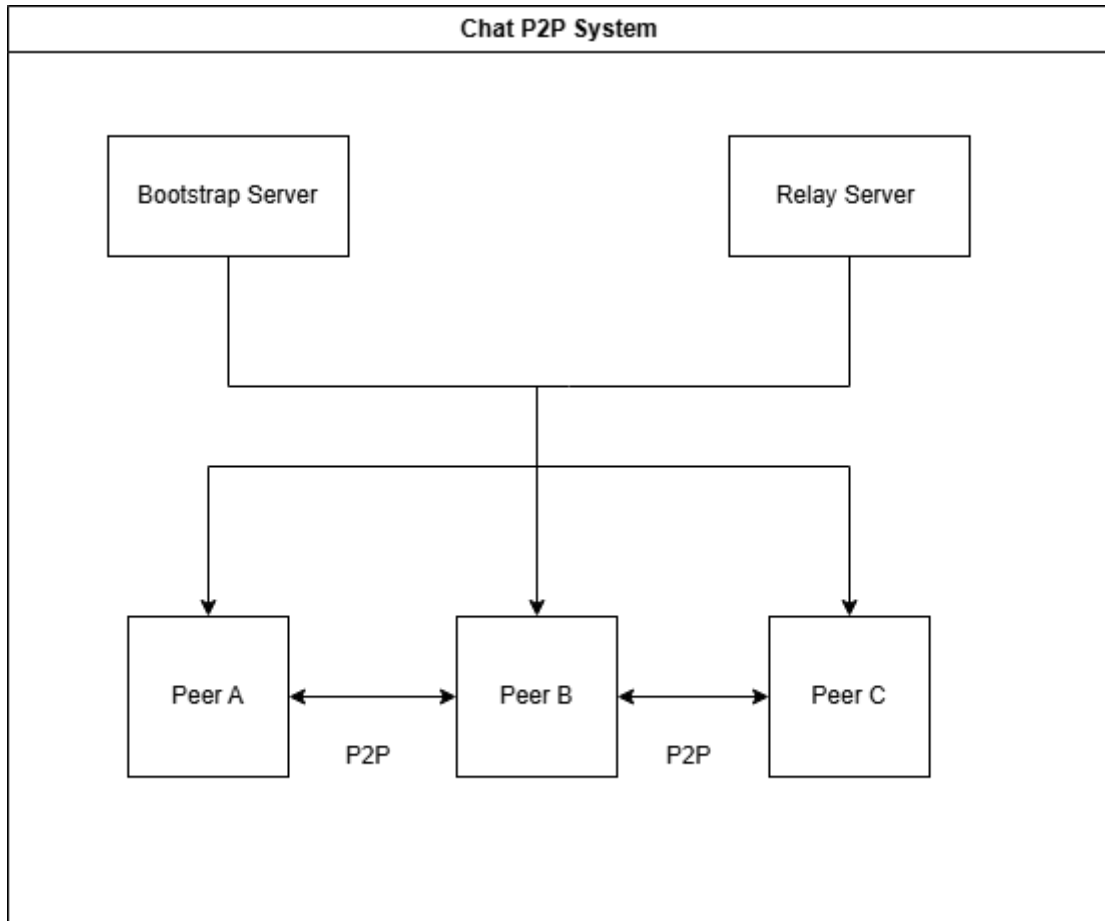
CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN

Đồ án xây dựng một hệ thống chat phân tán theo mô hình Peer-to-peer kết hợp. Không giống mô hình P2P thuần túy, hệ thống sử dụng hai máy chủ trung gian đóng vai trò hạ tầng hỗ trợ: một Bootstrap Server để quản lý sự tham gia và rời đi của các peer, và một Relay Server để lưu trữ và chuyển tiếp tin nhắn khi người nhận offline. Tin nhắn giữa các người dùng khi cả hai đều online được truyền trực tiếp P2P, không qua máy chủ trung gian.

CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG

2.1. Tổng quan kiến trúc

Hệ thống gồm 3 thành phần độc lập, mỗi thành phần là một module Maven riêng:



Hình 1: Kiến trúc tổng quan Chat P2P System

2.2. Các thành phần chính

Bootstrap Server:

Máy chủ trung tâm duy nhất biết toàn bộ topology của mạng. Chức năng chính là:

- Tiếp nhận đăng ký từ các peer mới.
- Trả về danh sách các peer đang online.
- Broadcast sự kiện tới toàn bộ mạng.
- Dọn dẹp các peer mất kết nối theo chu kỳ 30 giây.

- Lưu trạng thái các peer vào database.

Relay Server

Máy chủ hỗ trợ nhắn tin offline theo mô hình **Store-and-Forward**:

- Lưu tin nhắn gửi cho peer đang offline vào database.
- Thông báo cho peer đích ngay khi peer đó online trở lại.
- Tự động xóa tin nhắn sau 7 ngày.
- Hỗ trợ đếm số tin nhắn đang chờ.

Peer Client

Thành phần người dùng, đóng hai vai trò đồng thời:

- Server: mở một cổng TCP để lắng nghe kết nối từ các peer khác.
- Client: Chủ động kết nối tới Bootstrap, Relay và các peer khác.

2.3. Cơ sở dữ liệu

Hệ thống dùng 3 database MySQL độc lập, phân tách theo vai trò:

Database	Phục vụ	Bảng chính
p2pchat_bootstrap	Bootstrap Server	Users
p2pchat_relay	Relay Server	relay_message, registered_peers
p2chat_peer_{id}	Peer Client (local)	Messages, pending_messages, peer_groups, peer_group_members

Bảng 1: Bảng database hệ thống

Mỗi peer có database MySQL riêng trên máy của mình, lưu lịch sử chat và trạng thái nhóm. Đây là thiết kế phi tập trung về dữ liệu – Không có một nơi nào lưu toàn bộ lịch sử chat của mọi người.

2.4. Công nghệ sử dụng

Thành phần	Công nghệ	Phiên bản
Ngôn ngữ	Java	21
Build tool	Apache Maven	3.9+
Giao diện	Java Swing	Native
Database	MySQL	8.0+
Connection Pool	HikariCP	5.1.0
Logging	SLF4J + Simple	2.0.12
Testing	JUnit Jupiter	5.10.2

Bảng 2: Công nghệ sử dụng

CHƯƠNG 3: GIAO THỨC TRAO ĐỔI THÔNG ĐIỆN

3.1. Cơ chế truyền thông

Hệ thống sử dụng **TCP Socket** với **Java Object Serialization** (ObjectOutputStream / ObjectInputStream). Đây là giao thức tầng ứng dụng tự định nghĩa (custom binary protocol), không dùng HTTP hay WebSocket.

Mỗi message là một đối tượng Java (Message.java) được serialize và gửi trực tiếp qua socket. Class Message chứa:

- type: kiểu message (enum MessageType).
- senderPeerId, senderUsername: định danh người gửi.
- targetPeerId: peer đích (nếu có).
- content: nội dung tin nhắn (đã mã hóa AES-128 + Base64).
- meta: map lưu các thuộc tính bổ sung (danh sách peers, groupId, port, ...).
- messageId: UUID duy nhất dùng để chống xử lý trùng.
- timestamp: thời điểm gửi.

Lưu ý quan trọng trong code: tại ConnectionHandler.java, ObjectOutputStream được tạo **trước** ObjectInputStream và được flush() ngay lập tức để tránh deadlock khi hai đầu cùng khởi tạo stream đồng thời. Sau mỗi lần ghi, out.reset() được gọi để ngăn Java caching object references (tránh gửi bản cũ của object).

3.2. Bộ các loại message

Nhóm Bootstrap – Giao tiếp giữa Peer và Bootstrap Server:

Message	Chiều	Mô tả
REGISTER	Peer → Bootstrap	Đăng ký tham gia mạng, kèm peerID, username, port lắng nghe.
PEER_LIST	Bootstrap → Peer	Phản hồi danh sách peers đang online.
UNREGISTER	Peer → Bootstrap	Thông báo rời mạng có chủ ý.
PEER_JOINED	Bootstrap → All	Broadcast: có peer mới vào mạng
PEER_LEFT	Bootstrap → All	Broadcast: có peer rời mạng
HEARTBEAT	Peer → Peer / Bootstrap	Kiểm tra kết nối còn sống
HEARTBEAT_ACK	Bootstrap → Peer	Phản hồi heartbeat
GET_PEERS	Peer → Bootstrap	Lấy danh sách peers theo yêu cầu

Bảng 3: Message nhóm Bootstrap

Nhóm Chat P2P – Giao tiếp trực tiếp giữa các peer

Message	Mô tả
CHAT	Tin nhắn 1-1 trực tiếp
GROUP_CHAT	Tin nhắn nhóm, gửi tới từng thành viên
BROADCAST	Gửi tới toàn bộ mạng
ACK	Xác nhận đã nhận tin nhắn

Bảng 4: Message nhóm Chat P2P

Nhóm Group Management

Message	Mô tả
CREATE_GROUP	Tạo nhóm chat mới
JOIN_GROUP	Tham gia nhóm
LEAVE_GROUP	Rời nhóm
GROUP_INFO	Đồng bộ thông tin nhóm (danh sách thành viên)

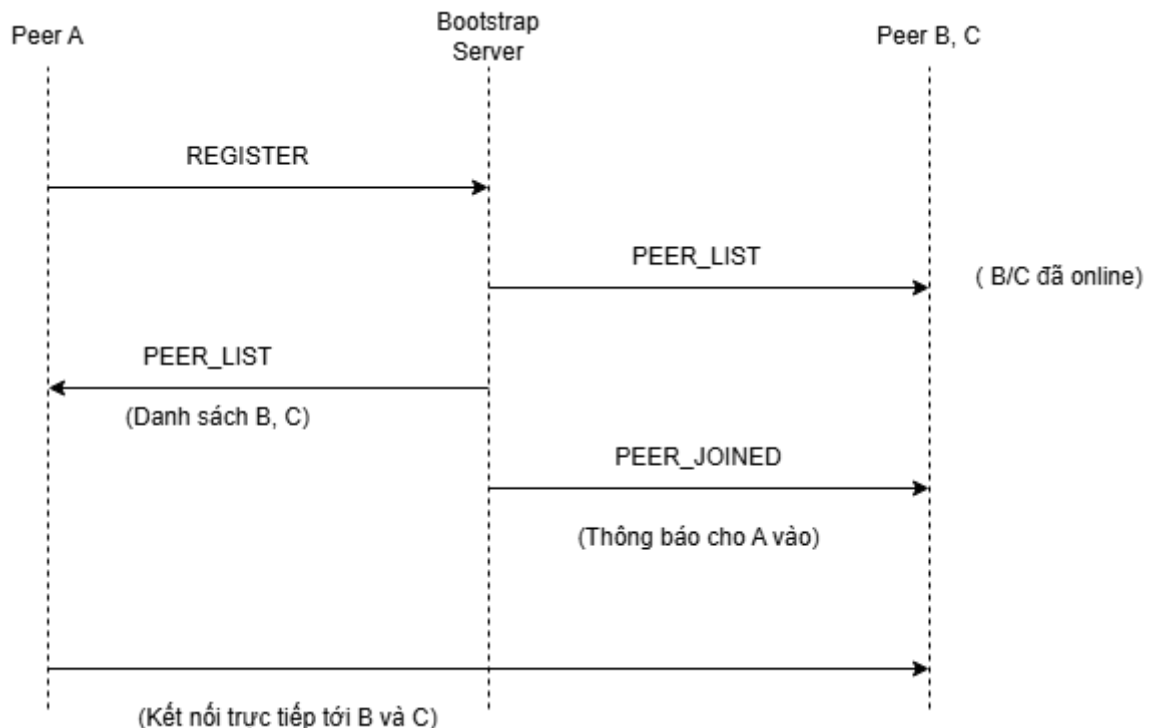
*Bảng 5: Message nhóm Group Management***Nhóm File Transfer**

Message	Mô tả
FILE_TRANSFER_REQUEST	Yêu cầu gửi file (kèm tên, kích thước)
FILE_TRANSFER_ACCEPT	Người nhận chấp nhận
FILE_TRANSFER_REJECT	Người nhận từ chối
FILE_CHUNK	Một chunk dữ liệu (64KB/chunk)
FILE_TRANSFER_COMPLETE	Hoàn thành truyền file
GROUP_FILE_REQUEST	Yêu cầu gửi file trong nhóm
GROUP_FILE_CHUNK	Chunk dữ liệu file nhóm
GROUP_FILE_COMPLETE	Hoàn thành file nhóm

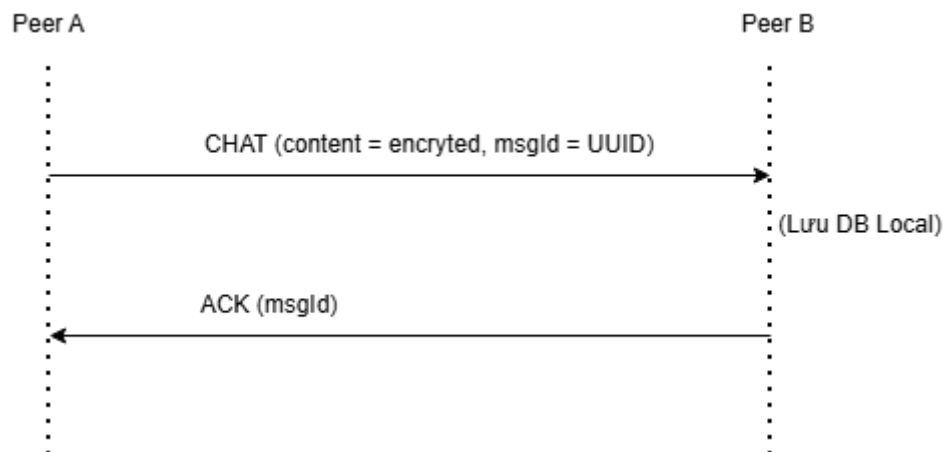
Bảng 6: Message nhóm File transfer

Nhóm Relay Server — tin nhắn offline

Message	Mô tả
RELAY_REGISTER	Peer đăng ký với relay server
RELAY_UNREGISTER	Peer rời relay server
STORE_MESSAGE	Gửi tin nhắn cần lưu trữ cho peer offline
FETCH_MESSAGES	Lấy toàn bộ tin nhắn đang chờ
DELETE_MESSAGE	Xóa tin nhắn đã được giao thành công
RELAY_MESSAGE_LIST	Relay trả về danh sách tin nhắn offline
STORE_NOTIFY	Relay thông báo có tin nhắn mới (push notification)
GET_PENDING_COUNT	Lấy số lượng tin nhắn đang chờ

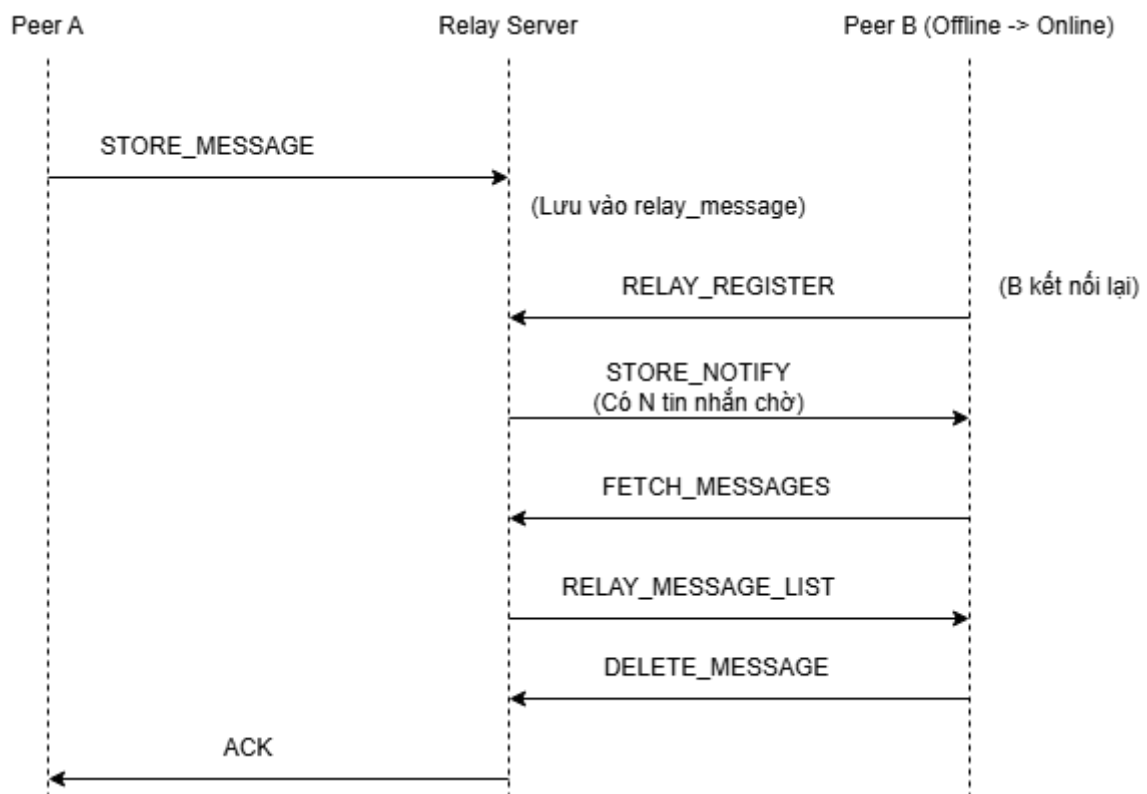
*Bảng 7: Message nhóm Relay Server***Nhóm System:** ERROR, PING, PONG.**3.3. Luồng trao đổi thông điệp chi tiết****Luồng Peer đăng ký vào mạng:***Hình 2: Luồng Peer đăng ký vào mạng*

Luồng chat khi cả hai online



Hình 3: Luồng chat khi cả hai online

Luồng tin nhắn offline (Store-and-Forward)



Hình 4: Luồng tin nhắn offline

3.4. Mã hóa nội dung

Nội dung tin nhắn được mã hóa bằng AES-128 với pre-shared key “P2PChatSecretKey”. Sau khi mã hóa, dữ liệu được encode Base64 trước khi đưa vào trường content của Message.

CHƯƠNG 4: CƠ CHẾ PEER DISCOVERY

4.1. Tổng quan

Hệ thống hỗ trợ hai phương thức để một peer tham gia vào mạng:

Mode 1 – Qua Bootstrap Server (mặc định): Peer kết nối tới Bootstrap Server (IP/Port cố định được cấu hình trước), gửi REGISTER, nhận lại PEER_LIST chứa thông tin toàn bộ peer đang online. Sau đó peer chủ động thiết lập kết nối TCP trực tiếp tới từng peer trong danh sách.

Mode 2 – Qua Peer đã biết: Dùng khi không có Bootstrap Server. Peer kết nối trực tiếp tới địa chỉ của một peer đã biết trước. Peer trung gian sẽ giúp forward thông tin vào mạng.

4.2. Quy trình Peer Discovery qua Bootstrap Server

Bước 1: Peer mở TCP server tại preferredPort. Nếu port bị chiếm, hệ thống tự tìm port khả dụng tiếp theo (PeerServer.findAvailablePort()).

Bước 2: Peer gửi REGISTER kèm {peerId, username, port}. Bootstrap Server xác nhận bằng PEER_LIST. Kết nối này được duy trì liên tục để nhận các event về sau.

Bước 3: Kết nối Bootstrap không đóng sau khi đăng ký. Thread bootstrap-listener tiếp tục chạy để nhận các thông báo PEER_JOINED (có peer mới vào) và PEER_LEFT (có peer rời đi). Khi nhận được PEER_JOINED, peer sẽ chủ động kết nối TCP trực tiếp tới peer mới đó.

Bước 4: Peer tạo ConnectionHandler cho mỗi peer trong danh sách, thiết lập kết nối TCP trực tiếp.

4.3. Duy trì thông tin topology

Để phát hiện peer chết không thông báo (crash, mất điện, mất mạng đột ngột), hệ thống dùng cơ chế **Heartbeat định kỳ**:

- Mỗi 15 giây, P2PNode gửi HEARTBEAT tới tất cả peer đang kết nối.
- ConnectionHandler phía nhận phản hồi HEARTBEAT_ACK ngay lập tức.
- Bootstrap Server chạy scheduler mỗi 30 giây (cleanDeadConnections()) để quét và xóa các handler không còn hoạt động khỏi danh sách online.

4.4. Chống xử lý tin nhắn trùng

Trong mạng P2P, tin nhắn có thể đến theo nhiều đường (ví dụ: cả trực tiếp lẫn qua relay). Hệ thống dùng cơ chế **deduplication window 2 phút**:



Hình 5: Mã chống xử lý tin nhắn trùng lặp

Mỗi messageId (UUID) được ghi lại với timestamp khi xử lý. Nếu cùng messageId đến trong vòng 2 phút, nó bị bỏ qua. Map này được dọn dẹp định kỳ.

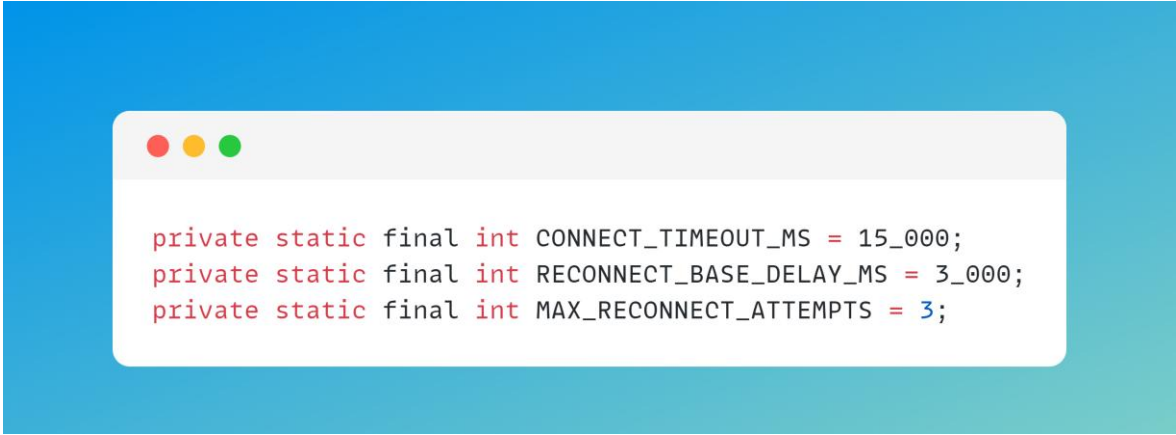
4.5. Kết nối Relay Server

Song song với Bootstrap, peer cũng kết nối tới Relay Server và gửi RELAY_REGISTER. Kết nối này cũng được duy trì liên tục để relay có thể push STORE_NOTIFY ngay khi có tin nhắn mới mà không cần peer polling.

CHƯƠNG 5: XỬ LÝ LỖI VÀ THỬ NGHIỆM HỆ THỐNG

5.1. Xử lý lỗi mạng — Tự động Reconnect

Cả BootstrapClient và RelayClient đều cài đặt cơ chế **tự động reconnect** với **exponential backoff**:



```
private static final int CONNECT_TIMEOUT_MS = 15_000;
private static final int RECONNECT_BASE_DELAY_MS = 3_000;
private static final int MAX_RECONNECT_ATTEMPTS = 3;
```

Hình 6: Mã config cơ chế tự động reconnect

Khi reconnect thành công, trạng thái được emit lên UI qua callback `onStatusChange`. Cơ chế **guard** đảm bảo `onStatusChange` chỉ được gọi khi trạng thái **thực sự thay đổi** (tránh spam event cùng trạng thái lên EDT của Swing).

Các loại lỗi được xử lý:

Tình huống	Xử lý
Bootstrap Server sập	Reconnect tự động, thông báo trạng thái lên UI
Peer mất kết nối đột ngột	ConnectionHandler catch EOFException/SocketException, gọi <code>onDisconnect</code>
Server gửi sai message type	Ném <code>IOException</code> với thông báo rõ ràng
<code>ClassNotFoundException</code> khi deserialize	Log warning, bỏ qua message không hợp lệ
Port bị chiếm	<code>PeerServer.findAvailablePort()</code> tự tìm port khác

Bảng 8: Các loại lỗi được xử lý

5.2. Xử lý lỗi database — Connection Pool

HikariCP được cấu hình với các timeout hợp lý:

Server	Max pool	Min idle	Connection Timeout
Bootstrap	20	5	10 giây
Peer	10	2	10 giây
Relay	20	5	10 giây

Bảng 9: HikariCP cấu hình timeout

Khi khởi động, Bootstrap Server gọi `userRepo.setAllOffline()` để reset trạng thái từ lần chạy trước — xử lý trường hợp server crash mà không kịp cleanup.

5.3. Xử lý lỗi truyền file

- File vượt giới hạn (50MB cho 1-1, 20MB cho nhóm) bị từ chối trước khi gửi.
- Người gửi có thể hủy truyền bằng `fileSendAborted` flag (`AtomicBoolean`).
- File nhận được lưu vào `receivedFilePayloads` (in-memory) chờ user bấm tải — tránh ghi file tự động.
- Truyền theo chunk 64KB: nếu kết nối đứt giữa chừng, lần truyền hiện tại thất bại nhưng không gây crash toàn bộ ứng dụng.

5.4. Unit Testing — BootstrapClientTest

Nhóm viết bộ test cho `BootstrapClient` sử dụng **mock TCP server** (tạo `ServerSocket` trên port ngẫu nhiên trong test) để kiểm tra mà không cần Bootstrap Server thật. Các test case:

Test 1: `connectAndRegister_success`

- Mô phỏng server gửi `PEER_LIST` hợp lệ.
- Kiểm tra: `connectAndRegister()` trả về danh sách peers, `client.isConnected() == true`.

Test 2: connectAndRegister_wrongResponse

- Server gửi PEER_JOINED thay vì PEER_LIST.
- Kiểm tra: ném IOException với thông báo phù hợp, client.isConnected() == false.

Test 3: run_exitsOnStop

- Kết nối thành công, rồi gọi stopReconnectLoop().
- Kiểm tra: thread bootstrap-listener kết thúc trong vòng 2 giây (listener.join(2000)).

Test 4: disconnect_works

- Gọi disconnect() sau khi kết nối.
- Kiểm tra: server nhận được UNREGISTER **trước** khi socket đóng (CountDownLatch), client.isConnected() == false.

Test 5: connectAndRegister_noSocketLeak

- Gọi connectAndRegister() hai lần liên tiếp.
- Kiểm tra: server nhận đúng 2 kết nối (không leak socket cũ), kết nối thứ 2 thành công.

Test 6: reconnect_firesStatusChangeOnceOnFailure

- Server từ chối mọi kết nối (đóng socket ngay sau accept()).
- Kiểm tra: sau khi hết MAX_RECONNECT_ATTEMPTS, callback onStatusChange(DISCONNECTED) được gọi đúng **một lần** (CountDownLatch).

5.5. Đánh giá hệ thống**Điểm mạnh:**

- Kiến trúc lai P2P + Server hợp lý: tận dụng lợi thế P2P (giảm tải server, truyền trực tiếp) trong khi vẫn có cơ sở hạ tầng hỗ trợ peer discovery và offline messaging.
- Thiết kế phi tập trung về dữ liệu: lịch sử chat nằm tại máy từng người dùng, không có điểm tập trung duy nhất bị lộ toàn bộ dữ liệu.
- Xử lý đa luồng tốt: dùng ConcurrentHashMap, AtomicBoolean, ExecutorService, ScheduledExecutorService đúng chỗ.

- Deduplication window ngăn xử lý trùng tin nhắn trong mạng P2P đa đường.
- Store-and-Forward 7 ngày đảm bảo không mất tin nhắn khi người nhận tạm thời offline.

Hạn chế và hướng cải thiện:

- Mã hóa AES-ECB với pre-shared key tĩnh: nên chuyển sang AES-GCM + Diffie-Hellman key exchange để đảm bảo forward secrecy và xác thực.
- Bootstrap Server là **điểm lỗi đơn (single point of failure)**: nếu Bootstrap sập, các peer mới không thể tham gia mạng. Có thể cải thiện bằng cách triển khai nhiều Bootstrap Server dự phòng hoặc tích hợp DHT (Distributed Hash Table).
- Chưa có NAT traversal: hệ thống giả định các peer có thể kết nối trực tiếp với nhau (phù hợp mạng LAN), nhưng sẽ gặp vấn đề khi peer ở sau NAT/firewall.
- ECB mode cho AES không che giấu pattern: các block plaintext giống nhau tạo ra ciphertext giống nhau.
- Thiếu xác thực peer: bất kỳ ai biết IP và port của Bootstrap đều có thể đăng ký vào mạng.

CHƯƠNG 6: KẾT LUẬN

Đồ án đã xây dựng thành công một hệ thống chat phân tán P2P với đầy đủ tính năng: chat 1-1, chat nhóm, broadcast, truyền file, và Store-and-Forward (tin nhắn offline). Hệ thống thể hiện các nguyên lý cốt lõi của hệ thống phân tán: **tính phân tán** (dữ liệu và xử lý không tập trung), **fault tolerance** (reconnect, deduplication, cleanup), **consistency** (đồng bộ thành viên nhóm, heartbeat), và **scalability** (connection pool, thread pool, P2P routing giảm tải server).

Bộ unit test với mock TCP server là điểm tốt — kiểm tra được các kịch bản lỗi thực tế mà không phụ thuộc vào hạ tầng bên ngoài. Các hạn chế về bảo mật (AES-ECB, pre-shared key) và khả năng chịu lỗi của Bootstrap Server là những bài toán mở cho phiên bản tiếp theo.